3 Flow Matching

In the previous section, we constructed flow and diffusion models as generative models parameterized by a neural network vector field u_t^θ. However, we have not yet discussed how to *train* them, i.e. how to optimize the parameters θ such that generative model returns something sensible, e.g. a nice-looking image or exciting video. Next, we discuss **flow matching** [25, 1, 27], a algorithm to train u_t^θ that is simple, scalable, and represents the current state-of-the-art.

In this section, we restrict ourselves to flow models, i.e. we have a neural network u_t^θ and obtain samples from the generative model by simulating the ODE

$$X_0 \sim p_{\text{init}}, \quad \mathrm{d}X_t = u_t^\theta(X_t)\mathrm{d}t \qquad \text{(Flow model)} \tag{10}$$

and using the endpoints X_1 fro $t = 1$ as samples. As we discussed, our goal is that X_1 is distributed according to the data distribution p_{data}, i.e. $X_1 \sim p_{\text{data}}$. Therefore, the question "how to train" the neural network is really the following question: **How do we optimize θ such that simulating the flow model in Equation (10) results in samples from the data distribution $X_1 \sim p_{\text{data}}$?**

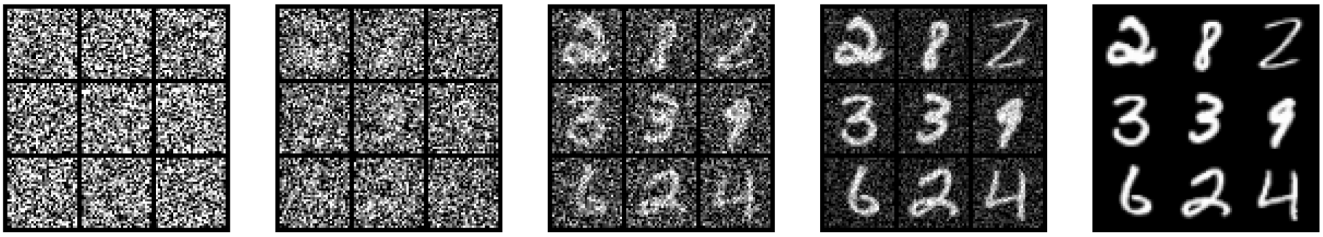


Figure 4: Gradual interpolation from noise to data via a Gaussian conditional probability path for a collection of images. Note that each image is a data point of dimension $d = 32 \times 32$, so we are plotting individual samples from the probability path, while in Figure 5 we plot the distribution as a 2d histogram.

3.1 Conditional and Marginal Probability Path

The first step of flow matching is to specify a **probability path**. Intuitively, a probability path specifies a gradual interpolation between noise p_{init} and data p_{data} (see Figure 4). But why would we want that? Remember that our desired ODE trajectory fulfills $X_0 \sim p_{\text{init}}$ for $t = 0$ and $X_1 \sim p_{\text{data}}$ for $t = 1$. But what about times $0 < t < 1$ in between start and end? It turns out that we have some freedom to choose what should happen in between and this is what is mathematically formalized in a probability path.

In the following, for a data point $z \in \mathbb{R}^d$, we denote with δ_z the **Dirac delta** "distribution". This is the simplest distribution that one can imagine: sampling from δ_z always returns z (i.e. it is deterministic). A **conditional (interpolating) probability path** is a set of distribution $p_t(x|z)$ over $\mathbb{R}^d$ such that:

$$p_0(\cdot|z) = p_{\text{init}}, \quad p_1(\cdot|z) = \delta_z \quad \text{for all } z \in \mathbb{R}^d. \tag{11}$$

In other words, a conditional probability path gradually converts the initial distribution p_{init} into a *single* data point (see e.g. Figure 4). You can think of a probability path as a trajectory in the space of distributions.

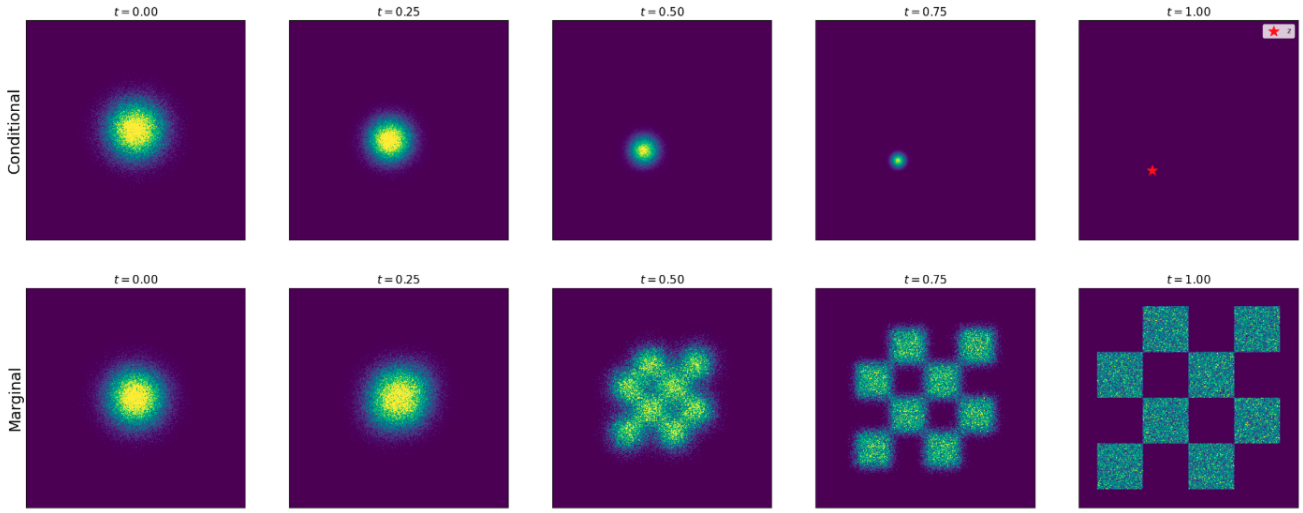


Figure 5: Illustration of a conditional (top) and marginal (bottom) probability path. Here, we plot a Gaussian probability path with $\alpha_t = t, \beta_t = 1 - t$. The conditional probability path interpolates a Gaussian $p_{\text{init}} = \mathcal{N}(0, I_d)$ and $p_{\text{data}} = \delta_z$ for single data point z. The marginal probability path interpolates a Gaussian and a data distribution p_{data} (Here, p_{data} is a toy distribution in dimension $d = 2$ represented by a chess board pattern.)

Every conditional probability path $p_t(x|z)$ induces a **marginal probability path** $p_t(x)$ defined as the distribution that we obtain by first sampling a data point $z \sim p_{\text{data}}$ from the data distribution and then sampling from $p_t(\cdot|z)$:

$$z \sim p_{\text{data}}, \quad x \sim p_t(\cdot|z) \quad \Rightarrow x \sim p_t \qquad \blacktriangleright \text{ sampling from marginal path} \tag{12}$$

$$p_t(x) = \int p_t(x|z) p_{\text{data}}(z) \mathrm{d}z \qquad \blacktriangleright \text{ density of marginal path} \tag{13}$$

Note that we know how to sample from p_t but we don't know the density values $p_t(x)$ as the integral is intractable (i.e. we can actually compute Equation (12) but not Equation (13)). Check for yourself that because of the conditions on $p_t(\cdot|z)$ in Equation (11), the marginal probability path p_t interpolates between p_{init} and p_{data}:

$$p_0 = p_{\text{init}} \quad \text{and} \quad p_1 = p_{\text{data}}. \qquad \blacktriangleright \text{ noise-data interpolation} \tag{14}$$

The - by far - most important example of a probability path is the Gaussian probability path - hence, we strongly recommend reading the next example thoroughly.

Example 8 (Gaussian Conditional Probability Path)
One particularly popular probability path is the **Gaussian probability path**. This is the **probability path used by most state-of-the-art models**. Let α_t, β_t be **noise schedulers**: two continuously differentiable, monotonic functions with $\alpha_0 = \beta_1 = 0$ and $\alpha_1 = \beta_0 = 1$. We then define the conditional probability path

$$p_t(\cdot|z) = \mathcal{N}(\alpha_t z, \beta_t^2 I_d) \qquad \blacktriangleright \text{ Gaussian conditional path} \tag{15}$$

which, by the conditions we imposed on α_t and β_t, fulfills

$$p_0(\cdot|z) = \mathcal{N}(\alpha_0 z, \beta_0^2 I_d) = \mathcal{N}(0, I_d), \quad \text{and} \quad p_1(\cdot|z) = \mathcal{N}(\alpha_1 z, \beta_1^2 I_d) = \delta_z,$$

where we have used the fact that a normal distribution with zero variance and mean z is just δ_z. Therefore, this choice of $p_t(x|z)$ fulfills Equation (11) for $p_{\text{init}} = \mathcal{N}(0, I_d)$ and is therefore a valid conditional interpolating path. In Figure 4, we illustrate its application to an image. We can express sampling from the marginal path p_t as:

$$z \sim p_{\text{data}},\ \epsilon \sim p_{\text{init}} = \mathcal{N}(0, I_d) \Rightarrow x = \alpha_t z + \beta_t \epsilon \sim p_t \qquad \blacktriangleright \text{ sampling from marginal Gaussian path} \tag{16}$$

Intuitively, the above procedure adds more noise for lower t until time $t = 0$, at which point there is only noise. In Figure 5, we plot an example of such an interpolating path.

3.2 Conditional and Marginal Vector Fields

A probability path $(p_t)_{0 \le t \le 1}$ specifies what distributions $X_t \sim p_t$ the points X_t along a trajectory *should* have. At this point, this is just what we "wish" to be the case. But how can we find a vector field such that the trajectories X_t follow the probability path? Flow matching explicitly constructs such a vector field - the "marginal vector field" - which we explain in this section.

For every data point $z \in \mathbb{R}^d$, let $u_t^{\text{target}}(\cdot|z)$ denote a **conditional vector field**. This can be any vector field such that corresponding ODE yields the conditional probability path $p_t(\cdot|z)$, i.e. such that it holds

$$X_0 \sim p_{\text{init}}, \quad \frac{\mathrm{d}}{\mathrm{d}t} X_t = u_t^{\text{target}}(X_t|z) \quad \Rightarrow \quad X_t \sim p_t(\cdot|z) \quad (0 \le t \le 1). \tag{17}$$

We can often find a conditional vector field $u_t^{\text{target}}(\cdot|z)$ analytically by hand (i.e. by just doing some algebra ourselves). We illustrate this by deriving a conditional vector field $u_t(x|z)$ for our running example of a Gaussian probability path in Example 10.

At first sight, a conditional vector field seems useless because all endpoints of the ODE X_1 will collapse to $X_1 = z$, i.e. we are just re-generating known data points z. However, the conditional vector field serves as a building block for a vector field that generates actual samples from p_{data}:

Theorem 9 (Marginalization trick)
Let $u_t^{\text{target}}(x|z)$ be a conditional vector field (Equation (17)). Then the **marginal vector field** $u_t^{\text{target}}(x)$ defined as

$$u_t^{\text{target}}(x) = \int u_t^{\text{target}}(x|z) \frac{p_t(x|z) p_{\text{data}}(z)}{p_t(x)} \mathrm{d}z, \tag{18}$$

follows the marginal probability path, i.e.

$$X_0 \sim p_{\text{init}}, \quad \frac{\mathrm{d}}{\mathrm{d}t} X_t = u_t^{\text{target}}(X_t) \quad \Rightarrow \quad X_t \sim p_t \quad (0 \le t \le 1). \tag{19}$$

In particular, $X_1 \sim p_{\text{data}}$ for this ODE, so that we might say "u_t^{target} converts noise p_{init} into data p_{data}".

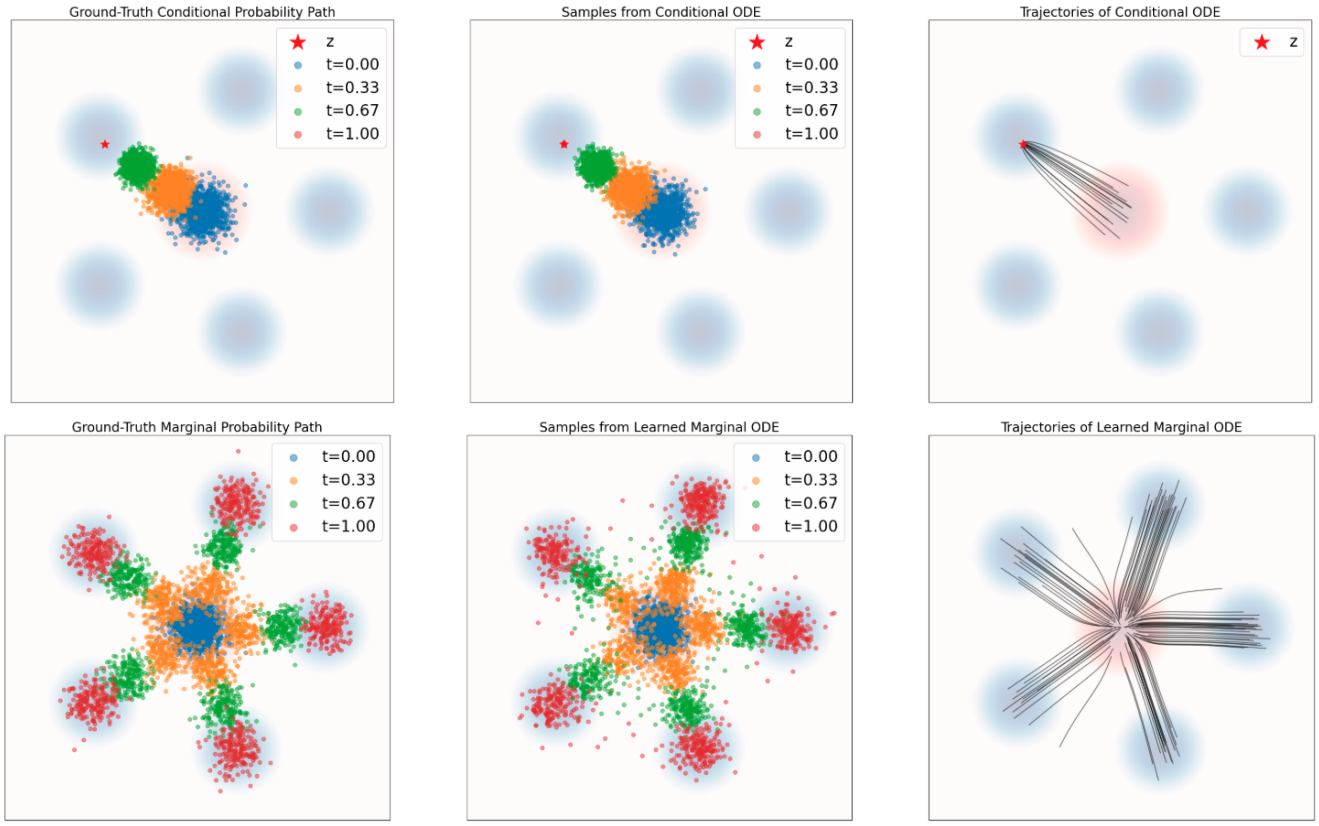


Figure 6: Illustration of Theorem 9. Simulating a probability path with ODEs. Data distribution p_{data} in blue background. Gaussian p_{init} in red background. Top row: Conditional probability path. Left: Ground truth samples from conditional path $p_t(\cdot|z)$. Middle: ODE samples over time. Right: Trajectories by simulating ODE with $u_t^{\text{target}}(x|z)$ in Equation (20). Bottom row: Simulating a marginal probability path. Left: Ground truth samples from p_t. Middle: ODE samples over time. Right: Trajectories by simulating ODE with marginal vector field $u_t^{\text{flow}}(x)$. As one can see, the conditional vector field follows the conditional probability path and the marginal vector field follows the marginal probability path.

Example 10 (Target ODE for Gaussian probability paths)
As before, let $p_t(\cdot|z) = \mathcal{N}(\alpha_t z, \beta_t^2 I_d)$ for noise schedulers α_t, β_t (see Equation (15)). Let $\dot{\alpha}_t = \partial_t \alpha_t$ and $\dot{\beta}_t = \partial_t \beta_t$ denote respective time derivatives of α_t and β_t. Here, we want to show that the **conditional Gaussian vector field** given by

$$u_t^{\text{target}}(x|z) = \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t} \alpha_t \right) z + \frac{\dot{\beta}_t}{\beta_t} x \tag{20}$$

is a valid conditional vector field model in the sense of Theorem 9: its ODE trajectories X_t satisfy $X_t \sim p_t(\cdot|z) = \mathcal{N}(\alpha_t z, \beta_t^2 I_d)$ if $X_0 \sim \mathcal{N}(0, I_d)$. In Figure 6, we confirm this visually by comparing samples from the conditional probability path (ground truth) to samples from simulated ODE trajectories of this flow. As you can see, the distribution match. We will now prove this.

Proof. Let us construct a conditional flow model $\psi_t^{\text{target}}(x|z)$ first by defining

$$\psi_t^{\text{target}}(x|z) = \alpha_t z + \beta_t x. \tag{21}$$

If X_t is the ODE trajectory of $\psi_t^{\text{target}}(\cdot|z)$ with $X_0 \sim p_{\text{init}} = \mathcal{N}(0, I_d)$, then by definition

$$X_t = \psi_t^{\text{target}}(X_0|z) = \alpha_t z + \beta_t X_0 \sim \mathcal{N}(\alpha_t z, \beta^2 I_d) = p_t(\cdot|z).$$

We conclude that the trajectories are distributed like the conditional probability path (i.e, Equation (17) is fulfilled). It remains to extract the vector field $u_t^{\text{target}}(x|z)$ from $\psi_t^{\text{target}}(x|z)$. By the definition of a flow (Equation (2b)), it holds

$$\begin{aligned}
&\frac{\mathrm{d}}{\mathrm{d}t}\psi_t^{\text{target}}(x|z) = u_t^{\text{target}}(\psi_t^{\text{target}}(x|z)|z) \quad \text{for all } x, z \in \mathbb{R}^d \\
\overset{(i)}{\Leftrightarrow} \quad &\dot{\alpha}_t z + \dot{\beta}_t x = u_t^{\text{target}}(\alpha_t z + \beta_t x|z) \quad \text{for all } x, z \in \mathbb{R}^d \\
\overset{(ii)}{\Leftrightarrow} \quad &\dot{\alpha}_t z + \dot{\beta}_t \left(\frac{x - \alpha_t z}{\beta_t}\right) = u_t^{\text{target}}(x|z) \quad \text{for all } x, z \in \mathbb{R}^d \\
\overset{(iii)}{\Leftrightarrow} \quad &\left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t}\alpha_t\right) z + \frac{\dot{\beta}_t}{\beta_t} x = u_t^{\text{target}}(x|z) \quad \text{for all } x, z \in \mathbb{R}^d
\end{aligned}$$

where in (i) we used the definition of $\psi_t^{\text{target}}(x|z)$ (Equation (21)), in (ii) we reparameterized $x \to (x - \alpha_t z)/\beta_t$, and in (iii) we just did some algebra. Note that the last equation is the conditional Gaussian vector field as we defined in Equation (20). This proves the statement.[a] □

[a] One can also double check this by plugging it into the continuity equation introduced later in this section.

See Figure 6 for an illustration of Theorem 9. Let's gain some intuition for the marginal vector field. **Bayes' rule** from statistics says that the following term describes a posterior distribution

$$\frac{p_t(x|z)p_{\text{data}}(z)}{p_t(x)} = \text{"posterior over data points } z \text{ given noisy data } x\text{"}$$

where $p_{\text{data}}(z)$ is the prior distribution. The marginal vector field then is simply a average: for every *possible* data point z it takes the velocity $u_t(x|z)$ - i.e. the direction that would bring us to z - and then weighs this velocity by how much we believe that x comes from z. Averaging over all data points, we obtain the marginal vector field.

The remainder of this section will make this intuition rigorous and prove Theorem 9. As the main mathematical tool, we will use the **continuity equation**, a fundamental equation in mathematics and physics. Define the **divergence** operator div as

$$\text{div}(v_t)(x) = \sum_{i=1}^{d} \frac{\partial}{\partial x_i} v_t^i(x) \tag{22}$$

where v_t^i is the i-th coordinate of v_t.

Theorem 11 (Continuity Equation)
Let us consider an flow model with vector field u_t^{target} with $X_0 \sim p_{\text{init}} = p_0$. Then $X_t \sim p_t$ for all $0 \leq t \leq 1$ if and only if

$$\partial_t p_t(x) = -\text{div}(p_t u_t^{\text{target}})(x) \quad \text{for all } x \in \mathbb{R}^d, 0 \leq t \leq 1, \tag{23}$$

where $\partial_t p_t(x) = \frac{\mathrm{d}}{\mathrm{d}t} p_t(x)$ denotes the time-derivative of $p_t(x)$. Equation 23 is known as the **continuity equation**.

For the mathematically-inclined reader, we present a self-contained proof of the Continuity Equation in Section B. Before we move on, let us try and understand intuitively the continuity equation. The left-hand side $\partial_t p_t(x)$ describes how much the probability $p_t(x)$ at x changes over time. Intuitively, the change should correspond to the net inflow of probability mass. For a flow model, a particle X_t follows along the vector field u_t^{target}. As you might recall from physics, the divergence measures a sort of net outflow from the vector field. Therefore, the negative divergence measures the net inflow. Scaling this by the total probability mass currently residing at x, we get that the net $-\text{div}(p_t u_t)$ measures the total inflow of probability mass. Since probability mass is conserved (always integrates to 1), the left-hand and right-hand side of the equation should be the same! We now proceed with a proof of the marginalization trick from Theorem 9.

Proof of Theorem 9. By Theorem 11, we have to show that the marginal vector field u_t^{target}, as defined as in Equation (18), satisfies the continuity equation. We can do this by direct calculation:

$$\begin{aligned}
\partial_t p_t(x) &\overset{(i)}{=} \partial_t \int p_t(x|z) p_{\text{data}}(z) \mathrm{d}z = \int \partial_t p_t(x|z) p_{\text{data}}(z) \mathrm{d}z \\
&\overset{(ii)}{=} \int -\text{div}(p_t(\cdot|z) u_t^{\text{target}}(\cdot|z))(x) p_{\text{data}}(z) \mathrm{d}z \\
&\overset{(iii)}{=} -\text{div}\left(\int p_t(x|z) u_t^{\text{target}}(x|z) p_{\text{data}}(z) \mathrm{d}z \right) \\
&\overset{(iv)}{=} -\text{div}\left(p_t(x) \int u_t^{\text{target}}(x|z) \frac{p_t(x|z) p_{\text{data}}(z)}{p_t(x)} \mathrm{d}z \right)(x) \\
&\overset{(v)}{=} -\text{div}\left(p_t u_t^{\text{target}} \right)(x),
\end{aligned}$$

where in (i) we used the definition of $p_t(x)$ in Equation (12), in (ii) we used the continuity equation for the conditional probability path $p_t(\cdot|z)$, in (iii) we swapped the integral and divergence operator using Equation (22), in (iv) we multiplied and divided by $p_t(x)$, and in (v) we used Equation (18). The beginning and end of the above chain of equations show that the continuity equation is fulfilled for u_t^{target}. By Theorem 11, this is enough to imply Equation (19), and we are done. □

3.3 Learning the Marginal Vector Field

Now, we are ready to describe the training algorithm. The goal of flow matching is to train the neural network u_t^θ such that it equals the marginal vector field u_t^{target}. If this holds, we know that the endpoints $X_1 \sim p_{\text{data}}$ have the desired distribution by Theorem 9. In the following, we denote by $\text{Unif} = \text{Unif}_{[0,1]}$ the uniform distribution on the interval $[0, 1]$, and by $\mathbb{E}$ the expected value of a random variable. An intuitive way of obtaining $u_t^\theta \approx u_t^{\text{target}}$ is to

use a mean-squared error, i.e. to use the **flow matching loss** defined as

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t\sim\text{Unif},x\sim p_t}[\|u_t^\theta(x) - u_t^{\text{target}}(x)\|^2] \tag{24}$$

$$\overset{(i)}{=} \mathbb{E}_{t\sim\text{Unif},z\sim p_{\text{data}},x\sim p_t(\cdot|z)}[\|u_t^\theta(x) - u_t^{\text{target}}(x)\|^2], \tag{25}$$

where $p_t(x) = \int p_t(x|z)p_{\text{data}}(z)\text{d}z$ is the marginal probability path and in (i) we used the sampling procedure given by Equation (12). Intuitively, this loss says: First, draw a random time $t \in [0,1]$. Second, draw a random point z from our data set, sample from $p_t(\cdot|z)$ (e.g., by adding some noise), and compute $u_t^\theta(x)$. Finally, compute the mean-squared error between the output of our neural network and the marginal vector field $u_t^{\text{target}}(x)$. Unfortunately, we are *not* done here. While we do know the formula for u_t^{target} by Theorem 9, we cannot compute it efficiently as the integral is intractable. Instead, we will exploit the fact that the **conditional** velocity field $u_t^{\text{target}}(x|z)$ is tractable. To do so, let us define the **conditional flow matching loss**

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t\sim\text{Unif},z\sim p_{\text{data}},x\sim p_t(\cdot|z)}[\|u_t^\theta(x) - u_t^{\text{target}}(x|z)\|^2]. \tag{26}$$

Note the difference to Equation (24): we use the conditional vector field $u_t^{\text{target}}(x|z)$ instead of the marginal vector $u_t^{\text{target}}(x)$. As we have an analytical formula for $u_t^{\text{target}}(x|z)$, we can minimize the above loss easily. But wait, what sense does it make to regress against the conditional vector field if it's the marginal vector field we care about? As it turns out, by *explicitly* regressing against the tractable, conditional vector field, we are *implicitly* regressing against the intractable, marginal vector field. The next result makes this intuition precise.

Theorem 12

The marginal flow matching loss equals the conditional flow matching loss up to a constant. That is,

$$\mathcal{L}_{\text{FM}}(\theta) = \mathcal{L}_{\text{CFM}}(\theta) + C,$$

where C is independent of θ. Therefore, their gradients coincide:

$$\nabla_\theta \mathcal{L}_{\text{FM}}(\theta) = \nabla_\theta \mathcal{L}_{\text{CFM}}(\theta).$$

Hence, minimizing $\mathcal{L}_{\text{CFM}}(\theta)$ with e.g., stochastic gradient descent (SGD) is equivalent to minimizing $\mathcal{L}_{\text{FM}}(\theta)$ in the same fashion. In particular, **for the minimizer θ^* of $\mathcal{L}_{\text{CFM}}(\theta)$, it will hold that $u_t^{\theta^*} = u_t^{\text{target}}$, i.e. the neural network will equal the marginal vector field** (assuming an infinitely expressive parameterization).

Direct Proof. The proof works by expanding the mean-squared error into three components and removing constants:

$$\begin{aligned}
\mathcal{L}_{\text{FM}}(\theta) &\overset{(i)}{=} \mathbb{E}_{t\sim\text{Unif},x\sim p_t}[\|u_t^\theta(x) - u_t^{\text{target}}(x)\|^2] \\
&\overset{(ii)}{=} \mathbb{E}_{t\sim\text{Unif},x\sim p_t}[\|u_t^\theta(x)\|^2 - 2u_t^\theta(x)^T u_t^{\text{target}}(x) + \|u_t^{\text{target}}(x)\|^2] \\
&\overset{(iii)}{=} \mathbb{E}_{t\sim\text{Unif},x\sim p_t}\left[\|u_t^\theta(x)\|^2\right] - 2\mathbb{E}_{t\sim\text{Unif},x\sim p_t}[u_t^\theta(x)^T u_t^{\text{target}}(x)] + \underbrace{\mathbb{E}_{t\sim\text{Unif}_{[0,1]},x\sim p_t}[\|u_t^{\text{target}}(x)\|^2]}_{=:C_1} \\
&\overset{(iv)}{=} \mathbb{E}_{t\sim\text{Unif},z\sim p_{\text{data}},x\sim p_t(\cdot|z)}[\|u_t^\theta(x)\|^2] - 2\mathbb{E}_{t\sim\text{Unif},x\sim p_t}[u_t^\theta(x)^T u_t^{\text{target}}(x)] + C_1
\end{aligned}$$

where (i) holds by definition, in (ii) we used the formula $\|a-b\|^2 = \|a\|^2 - 2a^Tb + \|b\|^2$, in (iii) we define a constant C_1 and in (iv) we used the sampling procedure of p_t given by Equation (12). Let us reexpress the second summand:

$$\begin{aligned}
\mathbb{E}_{t\sim\text{Unif},x\sim p_t}[u_t^\theta(x)^T u_t^{\text{target}}(x)] &\overset{(i)}{=} \int_0^1 \int p_t(x) u_t^\theta(x)^T u_t^{\text{target}}(x)\,\mathrm{d}x\,\mathrm{d}t \\
&\overset{(ii)}{=} \int_0^1 \int p_t(x) u_t^\theta(x)^T \left[\int u_t^{\text{target}}(x|z)\frac{p_t(x|z)p_{\text{data}}(z)}{p_t(x)}\mathrm{d}z\right]\mathrm{d}x\,\mathrm{d}t \\
&\overset{(iii)}{=} \int_0^1 \int\int u_t^\theta(x)^T u_t^{\text{target}}(x|z) p_t(x|z) p_{\text{data}}(z)\,\mathrm{d}z\,\mathrm{d}x\,\mathrm{d}t \\
&\overset{(iv)}{=} \mathbb{E}_{t\sim\text{Unif},z\sim p_{\text{data}},x\sim p_t(\cdot|z)}[u_t^\theta(x)^T u_t^{\text{target}}(x|z)]
\end{aligned}$$

where in (i) we expressed the expected value as an integral, in (ii) we use Equation (18), in (iii) we use the fact that integrals are linear, in (iv) we express the integral as an expected value. Note that this was really the crucial step of the proof: The beginning of the equality used the marginal vector field $u_t^{\text{target}}(x)$, while the end uses the conditional vector field $u_t^{\text{target}}(x|z)$. We plug is into the equation for $\mathcal{L}_{\text{FM}}$ to get:

$$\begin{aligned}
\mathcal{L}_{\text{FM}}(\theta) &\overset{(i)}{=} \mathbb{E}_{t\sim\text{Unif},z\sim p_{\text{data}},x\sim p_t(\cdot|z)}[\|u_t^\theta(x)\|^2] - 2\mathbb{E}_{t\sim\text{Unif},z\sim p_{\text{data}},x\sim p_t(\cdot|z)}[u_t^\theta(x)^T u_t^{\text{target}}(x|z)] + C_1 \\
&\overset{(ii)}{=} \mathbb{E}_{t\sim\text{Unif},z\sim p_{\text{data}},x\sim p_t(\cdot|z)}[\|u_t^\theta(x)\|^2 - 2u_t^\theta(x)^T u_t^{\text{target}}(x|z) + \|u_t^{\text{target}}(x|z)\|^2 - \|u_t^{\text{target}}(x|z)\|^2] + C_1 \\
&\overset{(iii)}{=} \mathbb{E}_{t\sim\text{Unif},z\sim p_{\text{data}},x\sim p_t(\cdot|z)}[\|u_t^\theta(x) - u_t^{\text{target}}(x|z)\|^2] + \underbrace{\mathbb{E}_{t\sim\text{Unif},z\sim p_{\text{data}},x\sim p_t(\cdot|z)}[-\|u_t^{\text{target}}(x|z)\|^2]}_{C_2} + C_1 \\
&\overset{(iv)}{=} \mathcal{L}_{\text{CFM}}(\theta) + \underbrace{C_2 + C_1}_{=:C}
\end{aligned}$$

where in (i) we plugged in the derived equation, in (ii) we added and subtracted the same value, in (iii) we used the formula $\|a-b\|^2 = \|a\|^2 - 2a^Tb + \|b\|^2$ again, and in (iv) we defined a constant in θ. This finishes the proof. □

Therefore, **flow matching training consists of minimizing the conditional flow matching loss.** The training procedure is summarized in Algorithm 3 and visualized in Figure 7. Note that there are several striking features about this algorithm: First, we never actually simulate any ODE during training. People call this feature of the algorithm **simulation-free**. This makes training extremely cheap as you don't have to roll out trajectories of the ODE during training (which takes a lot of steps). Second, the training is a simple regression objective - we are just regressing against $u_t^{\text{target}}(x|z)$. So it is not too different from supervised learning after all. Finally, the algorithm is extremely simple - it is hard to think of a much simpler training objective. All of this makes flow matching an extremely appealing method for large-scale machine learning models. Once u_t^θ has been trained, we may simulate the flow model

$$\mathrm{d}X_t = u_t^\theta(X_t)\,\mathrm{d}t, \qquad X_0 \sim p_{\text{init}} \tag{27}$$

via e.g., Algorithm 1 to obtain samples $X_1 \sim p_{\text{data}}$. This whole pipeline is called **flow matching** in the literature [25, 27, 1, 26]. Let us now instantiate the conditional flow matching loss for Gaussian probability paths:

Algorithm 3 Flow Matching Training Procedure (for Gaussian CondOT path $p_t(x|z) = \mathcal{N}(tz, (1-t)^2)$)

Require: A dataset of samples $z \sim p_{\text{data}}$, neural network u_t^θ

1: **for** each mini-batch of data **do**
2: Sample a data example z from the dataset.
3: Sample a random time $t \sim \text{Unif}_{[0,1]}$.
4: Sample noise $\epsilon \sim \mathcal{N}(0, I_d)$
5: Set

$$x = tz + (1-t)\epsilon \qquad \text{(General case: } x \sim p_t(\cdot \mid z)\text{)}$$

6: Compute loss

$$\mathcal{L}(\theta) = \|u_t^\theta(x) - (z - \epsilon)\|^2 \qquad \text{(General case: } = \|u_t^\theta(x) - u_t^{\text{target}}(x|z)\|^2\text{)}$$

7: Update $\theta \leftarrow \text{grad_update}(\mathcal{L}(\theta))$.
8: **end for**

Example 13 (Flow Matching for Gaussian Conditional Probability Paths)
Let us return to the example of Gaussian probability paths $p_t(\cdot|z) = \mathcal{N}(\alpha_t z; \beta_t^2 I_d)$, where we may sample from the conditional path via

$$\epsilon \sim \mathcal{N}(0, I_d) \quad \Rightarrow \quad x_t = \alpha_t z + \beta_t \epsilon \sim \mathcal{N}(\alpha_t z, \beta_t^2 I_d) = p_t(\cdot|z). \tag{28}$$

As we derived in Equation (20), the conditional vector field $u_t^{\text{target}}(x|z)$ is given by

$$u_t^{\text{target}}(x|z) = \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t}\alpha_t\right) z + \frac{\dot{\beta}_t}{\beta_t} x, \tag{29}$$

where $\dot{\alpha}_t = \partial_t \alpha_t$ and $\dot{\beta}_t = \partial_t \beta_t$ are the respective time derivatives. Plugging in this formula, the conditional flow matching loss reads

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t\sim\text{Unif}, z\sim p_{\text{data}}, x\sim\mathcal{N}(\alpha_t z, \beta_t^2 I_d)}[\|u_t^\theta(x) - \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t}\alpha_t\right) z - \frac{\dot{\beta}_t}{\beta_t} x\|^2] \tag{30}$$

$$\overset{(i)}{=} \mathbb{E}_{t\sim\text{Unif}, z\sim p_{\text{data}}, \epsilon\sim\mathcal{N}(0, I_d)}[\|u_t^\theta(\alpha_t z + \beta_t \epsilon) - (\dot{\alpha}_t z + \dot{\beta}_t \epsilon)\|^2] \tag{31}$$

where in (i) we plugged in Equation (28) and replaced x by $\alpha_t z + \beta_t \epsilon$. Note the simplicity of $\mathcal{L}_{\text{CFM}}$: We sample a data point z, sample some noise ϵ and then we take a mean squared error. Let us make this even more concrete for the special case of $\alpha_t = t$, and $\beta_t = 1 - t$. The corresponding probability $p_t(x|z) = \mathcal{N}(tz, (1-t)^2)$ is sometimes referred to as the (Gaussian) **CondOT probability path**. Then we have $\dot{\alpha}_t = 1, \dot{\beta}_t = -1$, so that

$$\mathcal{L}_{\text{cfm}}(\theta) = \mathbb{E}_{t\sim\text{Unif}, z\sim p_{\text{data}}, \epsilon\sim\mathcal{N}(0, I_d)}[\|u_t^\theta(tz + (1-t)\epsilon) - (z - \epsilon)\|^2]$$

Many famous state-of-the-art models have been trained using this simple yet effective procedure, e.g. *Stable Diffusion 3*, Meta's *Movie Gen Video*, and probably many more proprietary models. In Figure 7, we visualize

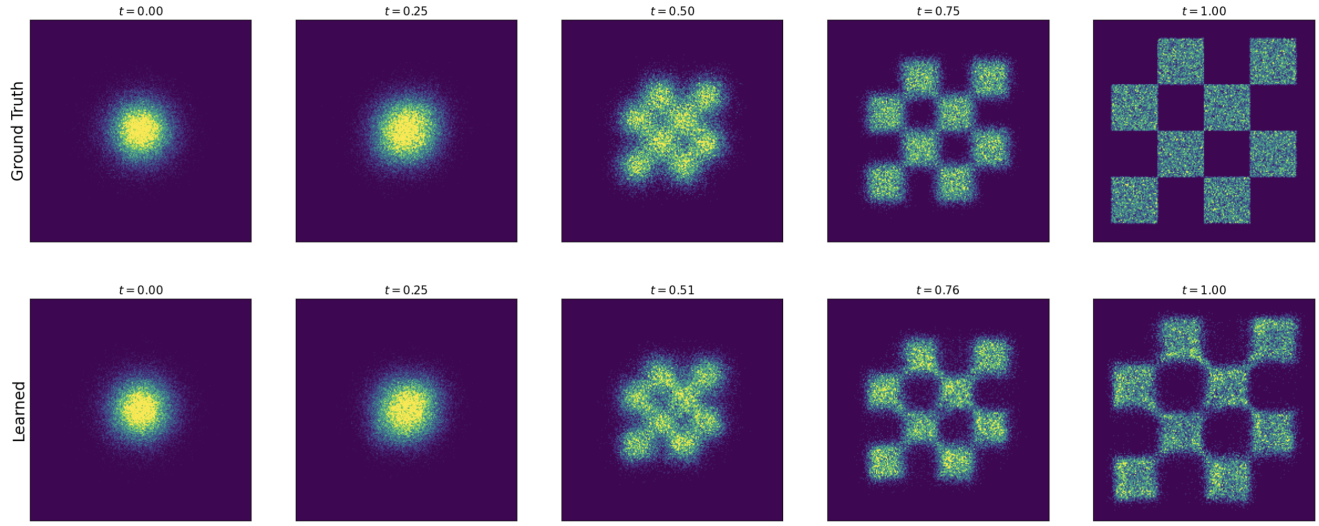


Figure 7: Illustration of Theorem 12 with a Gaussian CondOT probability path: simulating an ODE from a trained flow matching model. The data distribution is the chess board pattern (top right). Top row: Histogram from ground truth marginal probability path $p_t(x)$. Bottom row: Histogram of samples from flow matching model. As one can see, the top row and bottom row match after training (up to training error). The model was trained using Algorithm 3.

it in a simple example and in Algorithm 3 we summarize the training procedure.

Let us summarize the results of this section.

Summary 14 (Flow Matching)

Flow matching training consists of learning the marginal vector field u_t^{target}. To construct it, we choose a **conditional probability path** $p_t(x|z)$ that fulfils $p_0(\cdot|z) = p_{\text{init}}$, $p_1(\cdot|z) = \delta_z$. Next, we find a **conditional vector field** $u_t^{\text{target}}(x|z)$ such that its corresponding flow $\psi_t^{\text{target}}(x|z)$ fulfills

$$X_0 \sim p_{\text{init}} \quad \Rightarrow \quad X_t = \psi_t^{\text{target}}(X_0|z) \sim p_t(\cdot|z),$$

or, equivalently, that u_t^{target} satisfies the continuity equation. Then the **marginal vector field** defined by

$$u_t^{\text{target}}(x) = \int u_t^{\text{target}}(x|z) \frac{p_t(x|z)p_{\text{data}}(z)}{p_t(x)} \mathrm{d}z, \tag{32}$$

follows the marginal probability path, i.e.,

$$X_0 \sim p_{\text{init}}, \quad \mathrm{d}X_t = u_t^{\text{target}}(X_t)\mathrm{d}t \Rightarrow X_t \sim p_t \quad (0 \leq t \leq 1). \tag{33}$$

In particular, $X_1 \sim p_{\text{data}}$ for this ODE, so that u_t^{target} "converts noise into data", as desired. To learn it, we minimize the **conditional flow matching loss**

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t\sim\text{Unif}, z\sim p_{\text{data}}, x\sim p_t(\cdot|z)}[\|u_t^\theta(x) - u_t^{\text{target}}(x|z)\|^2]. \tag{34}$$

The most widely used example is the **Gaussian probability path**. For this case, the formulas become:

$$p_t(x|z) = \mathcal{N}(x; \alpha_t z, \beta_t^2 I_d) \tag{35}$$

$$u_t^{\text{flow}}(x|z) = \left(\dot{\alpha}_t - \frac{\dot{\beta}_t}{\beta_t}\alpha_t\right) z + \frac{\dot{\beta}_t}{\beta_t} x \tag{36}$$

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t\sim\text{Unif}, z\sim p_{\text{data}}, \epsilon\sim\mathcal{N}(0,I_d)}[\|u_t^\theta(\alpha_t z + \beta_t \epsilon) - (\dot{\alpha}_t z + \dot{\beta}_t \epsilon)\|^2] \tag{37}$$

for **noise schedulers** $\alpha_t, \beta_t \in \mathbb{R}$, i.e. continuously differentiable, monotonic functions that we choose such that $\alpha_0 = \beta_1 = 0$ $\alpha_1 = \beta_0 = 1$ (e.g. $\alpha_t = t, \beta_t = 1 - t$).